

RAG LECTURE ASSISTANT

A Retrieval-Augmented Generation web app for course lecture content

FastAPI

Streamlit

ChromaDB

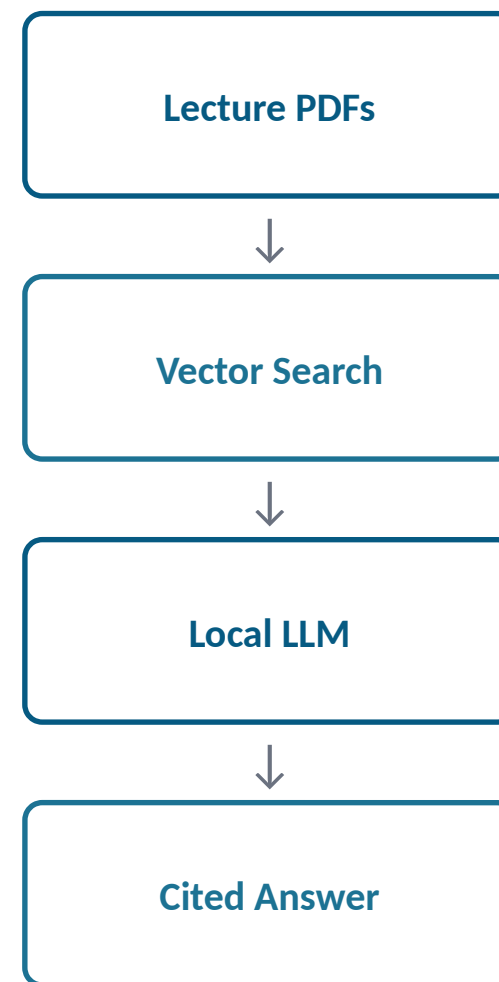
Ollama • llama3.2

What This Project Does

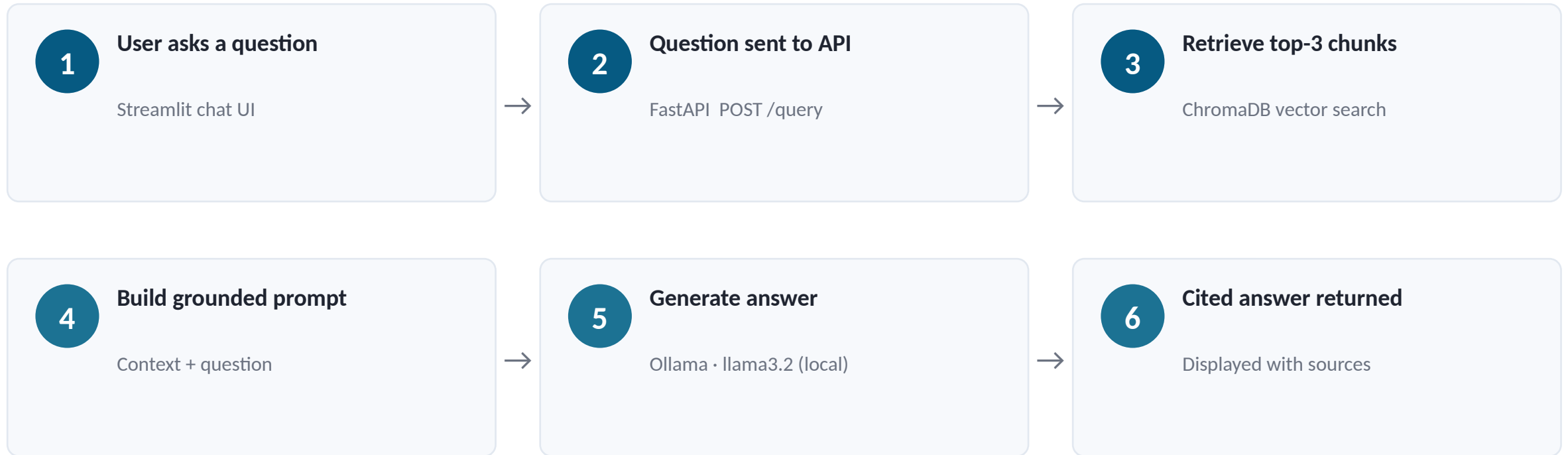
Lecture PDFs go in. Grounded, cited answers come out.

Instead of asking an LLM to answer from its own general training data, this system retrieves the exact lecture passages relevant to a question first, then asks the model to answer **only** from that retrieved content — citing the source file and page.

- P** **Problem** — General-purpose LLMs can answer confidently and be completely wrong about specific course content.
- A** **Approach** — Retrieval-Augmented Generation: search a vector database of lecture chunks, then generate from only what's retrieved.
- O** **Outcome** — A working chat app where every answer is traceable back to a real lecture file and page number.



System Architecture



Tech Stack

● LANGUAGE

Python 3.12

● EMBEDDINGS

sentence-transformers
all-MiniLM-L6-v2

● VECTOR STORE

ChromaDB (persistent, local)

● LLM

Ollama · llama3.2 (local)

● BACKEND API

FastAPI + Uvicorn

● FRONTEND

Streamlit

● PDF PARSING

pypdf

● TESTING

pytest + httpx

Building the RAG Pipeline

3

lecture PDFs

61

pages, 100% text-extractable

224

chunks (500 chars, 50 overlap)

384

dimensions per embedding

Notebook Phases (2.1 → 2.7)

- Load & Inspect — parse PDFs, verify extractable text
- Chunk — fixed-size (500 chars) with overlap for context continuity
- Embed & Store — sentence-transformers → ChromaDB (persisted to disk)
- Retrieve & Prompt — top-k similarity search + grounded prompt template
- Evaluate — 10 test questions, manually verified against source slides
- Export — config.json saved alongside the vector store for the backend

Backend & Frontend

Backend — FastAPI

REST API • localhost:8000/docs

- GET /health — server status check
- POST /query — retrieve → prompt → generate → return
- Vector store & embedding model loaded once at startup
- CORS enabled for the frontend's origin
- 2 tests: happy path + 422 on invalid input

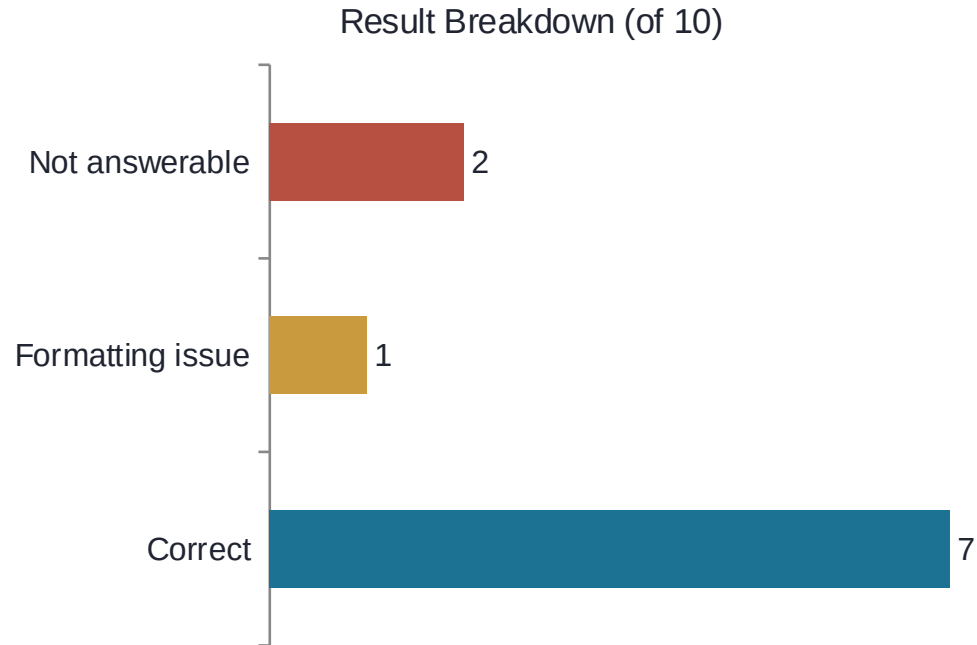
Frontend — Streamlit

Chat UI • localhost:8501

- Simple chat-style text input + Ask button
- Backend URL read from .env — never hard-coded
- Loading spinner while the request runs
- Friendly error message if the API is unreachable
- Displays the answer plus its cited sources

Evaluation Results

10 test questions, manually verified against the source lectures



● 7 correct & grounded

Accurate answers with proper page citations

● 1 formatting glitch

llama3.2 echoed the citation placeholder literally

● 2 correctly unanswered

Info wasn't in the slides — model said "I don't know" instead of guessing

Key finding: the system never fabricated an answer when the source content was missing — grounding worked as designed.

Key Takeaways

- A complete, working RAG pipeline: 3 lectures in, grounded cited answers out
- Runs fully locally — no API keys, no cloud costs (Ollama + open-source embeddings)
- Backend and frontend are decoupled, tested, and documented for reproducibility
- Honest evaluation: 7/10 correct, with transparent analysis of the 3 failure cases
- The system says "I don't know" rather than hallucinating — grounding works